

Chapter 21: RxJS

1. Overview

RxJS (Reactive Extensions for JavaScript) is the library Angular uses at its core for handling asynchronous streams of data: HTTP responses, router events, form value changes, user input, and its own change-detection-adjacent APIs (`async pipe`, `EventEmitter`, reactive forms `valueChanges` / `statusChanges`). If you use Angular, you use RxJS whether you want to or not — `HttpClient` returns `Observables`, the `Router` exposes `Observable` events, and reactive forms are built on top of `Subjects`.

The reason RxJS exists (rather than everyone using Promises) is that a **Promise represents a single future value**, while an **Observable represents a stream of values over time** — zero, one, or many — that can be transformed with a rich, composable set of operators before anyone subscribes to them. This makes RxJS the right tool for anything that is inherently a *stream*: keystrokes, WebSocket messages, mouse moves, interval timers, or a sequence of HTTP calls that must be cancelled, retried, throttled, or combined.

Interviewers use RxJS as a proxy for "does this person actually understand asynchronous Angular code, or have they just copy-pasted `.subscribe()` everywhere and never unsubscribed." This chapter is built around that reality: creation functions, the mapping-operator family (`switchMap` / `mergeMap` / `concatMap` / `exhaustMap`) which is the single most-asked RxJS interview topic, hot vs. cold semantics, and subscription memory management.

2. Core Concepts

2.1 Observable vs Promise

Aspect	Promise	Observable
Values emitted	Exactly one (resolve/reject)	Zero, one, or many, over time
Eagerness	Executes immediately on creation	Lazy — executor runs only on <code>subscribe()</code>
Cancellable	No (natively)	Yes — <code>unsubscribe()</code>
Operators	<code>.then()</code> , <code>.catch()</code> , <code>Promise.all/race</code>	Huge operator library (<code>map</code> , <code>filter</code> , <code>retry</code> , <code>debounceTime</code> , ...)
Multicast by default	N/A, single resolution is cached	No — cold by default, each subscriber gets its own execution unless multicast operators are used
Sync or async	Always async (microtask)	Can be sync or async depending on source
Error handling	<code>.catch()</code> , propagates through chain once	<code>catchError</code> operator, can retry the whole stream

```
// Promise: eager, single value
const promise = new Promise<number>((resolve) => {
  console.log('Promise executor runs immediately');
  resolve(42);
});
```

```
});

// Observable: lazy, nothing runs until subscribed
const obs$ = new Observable<number>((subscriber) => {
  console.log('Observable executor runs only when subscribed');
  subscriber.next(42);
  subscriber.complete();
});

obs$.subscribe(); // only now does the console.log fire
```

Key interview point: an Observable that is never subscribed to does *nothing at all*. This is why forgetting to `.subscribe()` on an `HttpClient` call is a classic bug — the request is never actually sent.

2.2 Observable Creation Functions

- `of(...values)` — emits the given arguments synchronously, one after another, then completes. Good for mocking/testing or emitting a fixed small set of values.
- `from(iterable | promise | array)` — converts an array, iterable, or Promise into an Observable that emits each element then completes. `from(promise)` emits once.
- `fromEvent(target, eventName)` — creates a stream from DOM/Node event emitters. Naturally hot (the DOM event fires whether or not you're listening).
- `interval(period)` — emits incrementing integers every `period` ms, forever, starting after the first tick (never emits immediately).
- `timer(delay, period?)` — emits once after `delay` ms; if `period` given, continues like `interval` afterward.
- `EMPTY` — completes immediately with no values. Useful as a "do nothing" branch inside `catchError` or `switchMap`.
- `throwError(() => error)` — creates an Observable that immediately errors. Used to synthesize errors inside operators.
- `Subject` / `BehaviorSubject` / `ReplaySubject` / `AsyncSubject` — covered in 2.4, these are both Observable *and* Observer, and are the primary way to create hot, multicast streams manually (e.g., event buses, state stores).

```
of(1, 2, 3).subscribe(console.log);           // 1, 2, 3 (sync)
from([1, 2, 3]).subscribe(console.log);       // 1, 2, 3
from(fetch('/api/x')).subscribe();            // resolves once
fromEvent(document, 'click').subscribe();     // hot, ongoing
interval(1000).subscribe();                   // 0,1,2,... every second
timer(2000, 1000).subscribe();                // wait 2s, then every 1s
```

2.3 Operators

Operators are pure functions that take an Observable as input and return a new Observable — they never mutate the source. They are composed with `.pipe()`.

Transformation

- `map(fn)` — transforms each emitted value, like `Array.prototype.map`. Synchronous, 1:1.

```
ages$.pipe(map(age => age + 1));
```

- `filter(predicate)` — drops values that don't match the predicate, like `Array.prototype.filter`.

```
numbers$.pipe(filter(n => n % 2 === 0));
```

- `tap(fn)` — performs a side effect (logging, analytics, dispatching) *without* modifying the stream. Passes every value through unchanged. Never put business logic that affects the returned value in `tap` — that's what `map` is for.

```
data$.pipe(tap(v => console.log('debug', v)));
```

The flattening / mapping-operator family (map + merge strategy)

These all take a project function that returns an *inner Observable* for each source value, and differ only in **how they manage overlapping inner subscriptions**. This is the single most important comparison in RxJS interviews.

- `mergeMap (alias flatMap)` — subscribes to *every* inner Observable as soon as the outer emits, and merges all their emissions concurrently, in the order they arrive. No cancellation, no ordering guarantee between overlapping inner streams.
 - Use when: multiple concurrent operations are all wanted and their results can interleave (e.g., uploading several files in parallel, each with its own progress stream).
 - Danger: unbounded concurrency can cause race conditions and out-of-order responses if the operations are not independent (e.g., firing save requests where an old one may resolve after a newer one and stomp its data).
- `concatMap` — subscribes to inner Observables **one at a time, in order**; a new inner Observable is not subscribed until the previous one completes. Guarantees order and completion, at the cost of concurrency (later requests are literally queued).
 - Use when: order matters and each request must fully finish before the next starts (e.g., a sequence of dependent PATCH calls, writing log entries in order, a queue of animations that must play sequentially).
 - Danger: if an early inner Observable hangs or never completes, everything behind it in the queue stalls forever.
- `switchMap` — subscribes to the new inner Observable and immediately **unsubscribes from ("switches away from") the previous** inner Observable, discarding whatever it hadn't emitted/completed yet.
 - Use when: only the most recent request matters and stale ones should be cancelled — typeahead/autocomplete search, route-param-driven data fetching (navigating away should cancel the in-flight request for the old param).
 - Danger: if used on a "submit" action (e.g., a save button), a second click cancels the first save's

HTTP call mid-flight, which is usually **not** what you want — this is the classic misuse that interviewers probe with the `exhaustMap` comparison.

- `exhaustMap` — subscribes to the first inner Observable and then **ignores all new outer emissions until the current inner Observable completes**. It never cancels an in-flight inner Observable, but it also never queues — anything that arrives while busy is dropped entirely.
 - Use when: you want to guard against duplicate submissions — login button, form submit, "load more" button — where a second click while the first request is still in flight should simply be ignored.
 - Danger: legitimate rapid actions get silently dropped if you use it somewhere concurrency or queuing was actually needed.

One-line mental model:

Operator	Concurrency	Order preserved	Cancels previous	Drops new while busy
<code>mergeMap</code>	Unlimited (or <code>concurrent</code> param)	No	No	No
<code>concatMap</code>	1 (queued)	Yes	No	No (queues instead)
<code>switchMap</code>	1 (latest wins)	N/A (only latest)	Yes	No (switches instead)
<code>exhaustMap</code>	1 (first wins)	N/A (only first)	No	Yes

Filtering / rate-limiting

- `debounceTime(ms)` — waits until the source has been silent for `ms` milliseconds, then emits only the most recent value. Resets the timer on every new emission. Ideal for search boxes: don't fire an HTTP call on every keystroke, wait until the user pauses.
- `throttleTime(ms)` — emits the first value, then ignores everything for the next `ms` milliseconds (by default). Different from `debounceTime`: throttle guarantees a leading emission and a fixed cadence; debounce waits for silence and can starve entirely if the user never stops typing.
- `distinctUntilChanged(compareFn?)` — suppresses consecutive duplicate emissions (compares to the *immediately previous* value only, not the whole history). Commonly chained after `debounceTime` in a search box so that re-typing the same string doesn't refire a request.

```
searchInput$.pipe(  
  debounceTime(300),  
  distinctUntilChanged(),  
  switchMap(term => this.api.search(term))  
);
```

- `take(n)` / `takeWhile(predicate)` / `first()` — complete the stream after `n` values / while a condition holds / after the first matching value.
- `takeUntil(notifier$)` — mirrors the source until `notifier$` emits, at which point it completes (and

unsubscribes from source). The standard Angular idiom for auto-unsubscribing is `ngOnDestroy`.

Error handling

- `catchError((err, caught$) => Observable)` — intercepts an error notification and replaces the erroring stream with a fallback Observable (e.g., `of(defaultValue)` or `EMPTY`), or rethrows via `throwError(() => err)`. Without `catchError`, an error terminates the subscription permanently (no more values, ever) — this is the #1 gotcha in Angular HTTP code, since one failed request can silently kill a shared, long-lived Observable pipeline if it's not scoped per request.
- `retry(count)` / `retry({ count, delay })` — resubscribes to the source Observable up to `count` times when it errors, effectively retrying the whole side-effecting operation (e.g., the HTTP call) from scratch.
- `retrywhen` (**deprecated in favor of** `retry({ delay: notifier })`) — allowed custom backoff logic by returning a notifier Observable that controls the retry timing.

```
this.http.get('/api/data').pipe(
  retry({ count: 3, delay: 1000 }),
  catchError(err => {
    this.logger.error(err);
    return of([]); // fallback so subscribers still complete gracefully
  })
);
```

Combination operators

- `combineLatest([a$, b$, c$])` — waits until *every* source has emitted at least once, then re-emits a combined array/tuple *every time any one source emits again*. Good for "recompute derived state whenever any dependency changes" (e.g., combine two filter dropdowns' selected values).
- `forkJoin({ a: a$, b: b$ })` — waits for **all** sources to *complete*, then emits a single combined result containing each source's *last* emitted value. This is the Observable equivalent of `Promise.all`. If any source never completes (e.g., an `interval`), `forkJoin` never emits. Ideal for "run N independent HTTP requests in parallel and proceed only once all have returned."
- `withLatestFrom(other$)` — on every emission of the *source*, samples the *latest* value from `other$` (without triggering on `other$`'s own emissions) and combines them. Useful when one stream is the trigger and another is just context/state to read at that moment (e.g., a "save" click stream combined with the latest form value).

```
// combineLatest: recompute whenever either changes
combineLatest([this.category$, this.sortOrder$]).pipe(
  map(([category, sort]) => this.buildQuery(category, sort))
);

// forkJoin: parallel, wait for all, like Promise.all
forkJoin({
  user: this.api.getUser(id),
  orders: this.api.getOrders(id),
}).subscribe(({ user, orders }) => { /* both are ready */ });
```

```
// withLatestFrom: trigger + snapshot of state
this.saveClick$.pipe(
  withLatestFrom(this.form.valueChanges),
  map(([, formValue]) => formValue)
).subscribe(v => this.api.save(v));
```

2.4 Hot vs Cold Observables

- **Cold** — the producer of values is created *inside* the Observable's subscriber function, so each new subscriber gets its own independent execution from the start. `HttpClient.get()`, `of()`, `interval()`, a freshly-constructed `new Observable(...)` are all cold: two subscribers to the same cold Observable trigger two separate HTTP requests / two separate timers, each seeing its own full sequence from the beginning.
- **Hot** — the producer exists independently of any subscriber (a DOM event, a WebSocket connection, a `Subject`), and subscribers just tap into whatever is currently happening — late subscribers miss past emissions. `fromEvent`, `Subject`, and any Observable passed through `share()` / `shareReplay()` / `multicast()` are hot.
- **Multicasting** turns a cold Observable hot by routing all subscribers through a single shared `Subject`. `share()` does this with reference counting (subscribes to source when first subscriber arrives, unsubscribes when last leaves); `shareReplay(n)` additionally caches the last `n` emissions and replays them to new/late subscribers — commonly used to cache an HTTP response so multiple components requesting the same data trigger only one network call.

```
// Cold: each subscribe = new HTTP call
const data$ = this.http.get('/api/data');
data$.subscribe(); // request #1
data$.subscribe(); // request #2 (separate!)

// Made hot/shared: only one HTTP call, replayed to late subscribers
const cached$ = this.http.get('/api/data').pipe(shareReplay(1));
cached$.subscribe(); // triggers the request
cached$.subscribe(); // reuses the same result, no new request
```

`Subject` variants, relevant because they're the building blocks of hot streams and Angular state services:

- `Subject` — no memory; late subscribers get nothing until the next `next()`.
- `BehaviorSubject(initialValue)` — always holds a current value, synchronously available via `.value`; new subscribers immediately receive the current value. Standard choice for simple state stores.
- `ReplaySubject(bufferSize)` — replays the last `bufferSize` emissions to new subscribers.
- `AsyncSubject` — only emits the *last* value, and only upon `complete()` — rarely used directly.

3. Code Examples

3.1 Typeahead search (`debounceTime` + `distinctUntilChanged` + `switchMap`)

```

import { Component, OnDestroy, OnInit } from '@angular/core';
import { FormControl } from '@angular/forms';
import { Subject } from 'rxjs';
import {
  debounceTime,
  distinctUntilChanged,
  switchMap,
  takeUntil,
  catchError,
  filter,
} from 'rxjs/operators';
import { of } from 'rxjs';
import { SearchService, SearchResult } from '../search.service';

@Component({
  selector: 'app-typeahead',
  template: `
    <input [formControl]="searchControl" placeholder="Search..." />
    <ul>
      <li *ngFor="let r of results">{{ r.title }}</li>
    </ul>
  `,
})
export class TypeaheadComponent implements OnInit, OnDestroy {
  searchControl = new FormControl('');
  results: SearchResult[] = [];
  private destroy$ = new Subject<void>();

  constructor(private searchService: SearchService) {}

  ngOnInit(): void {
    this.searchControl.valueChanges
      .pipe(
        filter((term): term is string => !!term && term.trim().length > 1),
        debounceTime(300), // wait for the user to pause typing
        distinctUntilChanged(), // skip re-searching the same term
        switchMap(term =>
          this.searchService.search(term).pipe(
            catchError(() => of([] as SearchResult[])) // per-request fallback
          )
        ),
        takeUntil(this.destroy$), // auto-unsubscribe on destroy
      )
      .subscribe(results => (this.results = results));
  }

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }
}

```

Why `switchMap` here: if the user types "an", then quickly "angular", the in-flight request for "an" is cancelled and only the "angular" response matters. Without `switchMap` (e.g., using `mergeMap`), a slow "an" response arriving after the fast "angular" response would overwrite the newer, more relevant results — a race condition.

3.2 Submit-guard (`exhaustMap`)

```
import { Component } from '@angular/core';
import { Subject } from 'rxjs';
import { exhaustMap, tap, catchError } from 'rxjs/operators';
import { of } from 'rxjs';
import { OrderService } from './order.service';

@Component({
  selector: 'app-checkout',
  template: `<button (click)="submit$.next()" [disabled]="submitting">Place Order</button>`,
})
export class CheckoutComponent {
  submit$ = new Subject<void>();
  submitting = false;

  constructor(private orderService: OrderService) {
    this.submit$
      .pipe(
        tap(() => (this.submitting = true)),
        exhaustMap(() =>
          this.orderService.placeOrder().pipe(
            catchError(err => {
              console.error('Order failed', err);
              return of(null);
            })
          )
        ),
        tap(() => (this.submitting = false)),
      )
      .subscribe(result => {
        if (result) this.onOrderPlaced(result);
      });

    private onOrderPlaced(result: unknown): void {
      // navigate to confirmation, etc.
    }
  }
}
```

Why `exhaustMap` here: rapid double-clicks on "Place Order" must not create two orders. Every click while the first `placeOrder()` call is still in flight is silently ignored until it completes — no cancellation, no queueing, no duplicate submission. `switchMap` would be wrong here (it would cancel the first in-flight order on the second click, an inconsistent server state); `mergeMap` / `concatMap` would both eventually submit twice.

3.3 Sequential dependent requests (`concatMap`)


```
import { from } from 'rxjs';
import { concatMap, toArray } from 'rxjs/operators';
import { InvoiceService } from './invoice.service';

export function sendInvoicesInOrder(invoiceIds: string[], invoiceService: InvoiceService) {
  return from(invoiceIds).pipe(
    concatMap(id => invoiceService.sendInvoice(id)), // waits for each send to finish before
the next
    toArray(), // collect all results once every request has completed, in original order
  );
}

// usage
sendInvoicesInOrder(['inv-1', 'inv-2', 'inv-3'], invoiceService).subscribe(results => {
  console.log('All invoices sent, in order:', results);
});
```

Why `concatMap` here: invoices must be sent one at a time, in the given order — perhaps because the backend enforces a sequential numbering constraint, or to avoid overwhelming a rate-limited endpoint. `mergeMap` would fire all three requests concurrently (breaking ordering guarantees and potentially hitting rate limits); `switchMap` would cancel `inv-1`'s send as soon as `inv-2` is queued, which is nonsensical for a fire-and-forget-but-ordered batch.

4. Internal Working

4.1 The Observer pattern under the hood

An Observable is, at its simplest, a wrapper around a function that receives an **Observer** (an object with `next`, `error`, `complete` methods) and returns a **teardown function** (or `Subscription`-like object). A minimal reimplementation:

```
class MiniObservable<T> {
  constructor(private subscribeFn: (observer: {
    next: (v: T) => void;
    error: (e: any) => void;
    complete: () => void;
  }) => (() => void) | void) {}

  subscribe(observer: {
    next?: (v: T) => void;
    error?: (e: any) => void;
    complete?: () => void;
  }) {
    let closed = false;
    const safeObserver = {
      next: (v: T) => { if (!closed) observer.next?.(v); },
      error: (e: any) => { if (!closed) { closed = true; observer.error?.(e); } },
      complete: () => { if (!closed) { closed = true; observer.complete?.(); } },
    };
    const teardown = this.subscribeFn(safeObserver) ?? (() => {});
    return {

```

```

        unsubscribe: () => { closed = true; teardown(); },
    };
}
}

```

Every `subscribe()` call independently re-invokes `subscribeFn` — this is *why cold Observables re-run their producer per subscriber*: there is no shared state between calls unless you deliberately introduce one (a `Subject` in between, i.e., multicasting).

4.2 How operators compose via `pipe()`

Each operator (e.g., `map(fn)`) is a factory function that returns a function `(source: Observable<T>) => Observable<R>`. `.pipe(op1, op2, op3)` is just left-to-right function composition:

```

observable.pipe(op1, op2, op3)
// is equivalent to
op3(op2(op1(observable)))

```

Internally, `map(project)` returns a new Observable whose `subscribeFn` subscribes to the *source*, and wraps the Observer it gets so that every `next(value)` becomes `next(project(value))` before being forwarded — errors and completion pass through unchanged. Every operator follows this same shape: wrap the source, wrap the downstream Observer, forward modified notifications. This is why operators compose cleanly: each one is just a thin proxy layer around the previous one, and subscribing to the outermost Observable cascades a chain of `subscribe()` calls down to the original source.

4.3 Subscription teardown

`subscribe()` returns a `Subscription` object with `unsubscribe()`. Calling it invokes the teardown logic registered by the source (e.g., clearing a `setInterval`, aborting an HTTP request, removing a DOM event listener) and — critically — propagates up through every operator in the chain, since each operator's own subscription to its source is itself torn down when the downstream unsubscribes.

`Subscription.add(otherSub)` lets you group multiple subscriptions so a single `unsubscribe()` cleans all of them up.

`takeUntil(notifier$)` works by subscribing to both the source and the notifier; when the notifier emits, it calls `complete()` on the downstream observer *and* unsubscribes from the source — this is the mechanically correct way "notifier-driven teardown" happens, and it's why `takeUntil` must be the **last** operator in a pipe chain (if placed before an operator like `switchMap`, later operators wouldn't be torn down by the notifier).

4.4 Hot vs cold, mechanically

A cold Observable's `subscribeFn` closure *creates the producer inside itself* (e.g., calls `XMLHttpRequest` fresh, or `setInterval` fresh) — so N subscribers = N independent producers. A hot Observable/Subject instead holds an internal array of registered Observers; `next(value)` just loops over that array and calls `.next(value)` on each currently-registered Observer. The producer (e.g., a WebSocket, a DOM listener) is created once, outside/independently of any individual `subscribe()` call, and multiple Observers simply share whatever that single producer emits — hence late subscribers miss earlier emissions (unless the Subject buffers, like `ReplaySubject`).

5. Edge Cases & Gotchas

- **Memory leaks from unmanaged subscriptions.** If a component subscribes to a long-lived Observable (an `interval`, a `Subject` in a shared service, `router.events`) and never unsubscribes, the subscription — and the component instance it closes over — is kept alive by the source even after the component is destroyed and removed from the DOM. This is a genuine memory leak and can also cause "zombie" side effects (a destroyed component's callback still firing, updating detached UI, or writing to a service). Fix with `takeUntil(this.destroy$)`, the `async` pipe (which auto-unsubscribes), or Angular's `takeUntilDestroyed()` (Angular 16+, from `@angular/core/rxjs-interop`).
- **HttpClient Observables are the exception, mostly.** A plain `http.get()` completes after one emission, so forgetting to unsubscribe rarely leaks — *but* if it's piped through `retry()`, `switchMap` from a long-lived source, or combined with `shareReplay()` without a `refCount`, it can still stay open indefinitely.
- **switchMap cancelling in-flight work that shouldn't be cancelled.** Using `switchMap` for a save/submit action can silently abort a real HTTP write mid-flight when the user clicks again quickly, leaving the server in an ambiguous state (was it saved or not?). This is the most common "clever code, wrong operator" bug reported in production Angular apps.
- **mergeMap causing race conditions.** Because inner Observables run concurrently with no ordering guarantee, a slower earlier request resolving *after* a newer one can overwrite fresher state with stale data. Bound concurrency with `mergeMap(project, concurrent)` if you need some parallelism but not unlimited.
- **concatMap causing head-of-line blocking.** If one item in a `concatMap` queue never completes (a hung request with no timeout), every subsequent queued item waits forever. Always pair with `timeout()` when queuing external calls.
- **exhaustMap silently dropping legitimate actions.** If the "busy" inner Observable takes unexpectedly long, additional valid user actions are dropped with zero feedback unless you also disable the triggering UI element or show a loading state (as in 3.2's `submitting` flag).
- **Nested subscribe() calls ("subscribe hell").** Subscribing inside another `subscribe()` callback to sequence two async calls is a classic anti-pattern — it defeats operator composition, makes error handling and cancellation nearly impossible, and is the RxJS equivalent of callback pyramid-of-doom.

```
// Anti-pattern
this.userService.getUser(id).subscribe(user => {
  this.orderService.getOrders(user.id).subscribe(orders => {
    this.render(user, orders); // no unsubscribe, no shared error handling
  });
});

// Correct: flatten with switchMap/mergeMap and combine cleanly
this.userService.getUser(id).pipe(
  switchMap(user => this.orderService.getOrders(user.id).pipe(
    map(orders => ({ user, orders })))
),
  takeUntil(this.destroy$),
).subscribe(({ user, orders }) => this.render(user, orders));
```

- **One error kills the whole shared stream.** If `catchError` is missing and a shared/multicast Observable (e.g., a `BehaviorSubject`-backed state stream) has an operator that throws, the *entire* stream terminates for *every* subscriber, permanently — not just the current operation. Always scope `catchError` around the specific risky sub-stream (typically inside the projection function of a mapping operator), not just at the very end of the whole pipeline, if the outer stream must survive individual failures.
- **`forkJoin` with a never-completing source.** If any input Observable to `forkJoin` never completes (e.g., accidentally passing an `interval()` instead of `take(1)`-limited stream), `forkJoin` never emits at all, and callers waiting on it hang silently with no error.
- **`combineLatest` before all sources have emitted.** `combineLatest` produces nothing until *every* input has emitted at least once — a `BehaviorSubject` guarantees an immediate value, but a plain `Subject` or a cold Observable that hasn't fired yet will stall the whole combination indefinitely.
- **`async` pipe subscribing multiple times.** Using the same Observable with `| async` twice in a template subscribes twice — for a cold Observable, this means two separate HTTP calls/executions. Fix by assigning to a local template variable via `*ngIf="data$ | async as data"`, or by applying `shareReplay(1)` to the source.

6. Interview Questions & Answers

Q1. What is an Observable, in one sentence?

A: A lazy, cancellable, push-based collection of values delivered over time to any number of Observers, that does nothing until `subscribe()` is called.

Q2. What are the fundamental differences between a Promise and an Observable?

A: A Promise represents exactly one future value, executes eagerly the moment it's created, and cannot be cancelled. An Observable can emit zero, one, or many values over time, is lazy (its producer function only runs on `subscribe()`), is cancellable via `unsubscribe()`, and comes with a large composable operator library (`map`, `retry`, `debounceTime`, etc.) that Promises don't have natively.

Interviewer intent: Checking whether the candidate understands *laziness* specifically — many candidates only mention "multiple values" and miss that an unsubscribed Observable never runs its side effects at all, which explains real bugs like "I called `http.get()` but nothing happened."

Q3. What's the difference between `of(1,2,3)` and `from([1,2,3])`?

A: `of` treats its arguments as the individual values to emit — `of([1,2,3])` would emit the whole array as one value. `from` takes a single iterable/array/Promise/Observable-like and flattens it into individual emissions — `from([1,2,3])` emits `1`, then `2`, then `3`. `from(promise)` emits the promise's resolved value once and completes.

Q4. Explain `map` vs `tap`. Why shouldn't you mutate state inside `map`?

A: `map` transforms each emitted value and its return value *becomes* the new emission passed downstream. `tap` performs a side effect and passes the original value through unchanged — its return value is ignored. Mutating external state inside `map` conflates transformation with side effects, making the pipeline harder to reason about and test; side effects belong in `tap` (or, better, in the `subscribe()` callback) so the data-transformation stages stay pure.

Q5. What does `debounceTime` do differently from `throttleTime`?

A: `debounceTime(ms)` waits for `ms` of silence after the last emission before emitting the latest value — it resets its timer on every new emission, so a continuously-firing source (someone who never stops typing) could theoretically starve it forever. `throttleTime(ms)` emits immediately on the first value, then ignores everything for the next `ms`, guaranteeing a value at a predictable cadence regardless of continued input.

Q6. Walk through `switchMap` vs `mergeMap` vs `concatMap` vs `exhaustMap` — compare their concurrency and cancellation behavior, and give a real use case for each.

A:

- `switchMap`: cancels the previous inner Observable the moment a new outer value arrives; only the latest matters. Use case: typeahead search — stale in-flight requests should be discarded.
- `mergeMap`: subscribes to every inner Observable concurrently, no cancellation, results interleave in arrival order. Use case: uploading multiple independent files in parallel.
- `concatMap`: queues inner Observables, only subscribing to the next once the current one completes — preserves order, no concurrency. Use case: a sequence of dependent writes that must happen in order.
- `exhaustMap`: ignores every new outer value while an inner Observable is still active; nothing is queued or cancelled, it's simply dropped. Use case: guarding a submit button against duplicate clicks.

Interviewer intent: This is the single most common RxJS interview question. The interviewer is testing whether the candidate can reason about *concurrency and cancellation semantics*, not just recite definitions — a strong answer ties each operator to a concrete bug it would cause if swapped for another (e.g., "using `switchMap` on a save button cancels the in-flight save").

Q7. Why is `distinctUntilChanged()` usually paired with `debounceTime()` in a search box, and what does it NOT protect against?

A: After debouncing, the user might still retype the exact same search term as before their pause (e.g., typed "cat", deleted, retyped "cat"). `distinctUntilChanged()` compares against only the *immediately previous* emitted value and skips re-emitting if it's equal, preventing a redundant API call. It does **not** protect against the same value reappearing non-consecutively (e.g., "cat" → "dog" → "cat" would fire twice) since it only compares to the last value, not the full history.

Q8. What's the difference between `catchError` and `retry`?

A: `retry(n)` resubscribes to the *source* Observable up to `n` times when it errors, essentially re-running the whole operation (e.g., re-sending the HTTP request) hoping the error was transient. `catchError` intercepts the error notification and *replaces* the stream with a fallback Observable (e.g., `of(defaultValue)` or `EMPTY`) — it doesn't retry anything, it gracefully degrades. They're often combined: `retry(3)` then `catchError` to provide a final fallback if all retries are exhausted.

Q9. What happens to a subscription if an operator throws an unhandled error, on a stream shared by multiple subscribers?

A: The error notification propagates downstream to every current subscriber and the Observable terminates permanently — a stream that has errored cannot emit further values even if the underlying condition resolves; a brand-new subscription (and, for cold sources, a fresh producer run) is required to try again. This is especially dangerous for long-lived multicast sources like a `BehaviorSubject`-backed state stream, where an unhandled error can silently kill state updates app-wide.

Interviewer intent: Testing whether the candidate understands that RxJS errors are *terminal* by default (unlike a caught exception in imperative code) — a frequent real-world bug is a shared service Observable dying permanently after one HTTP failure and never recovering, because `catchError` was placed too late or omitted.

Q10. What is the difference between `forkJoin` and `combineLatest`?

A: `forkJoin` waits for all input Observables to *complete*, then emits once with each one's *last* value — behaving like `Promise.all`. It never emits more than once, and never emits at all if any source never completes. `combineLatest` doesn't require completion — it emits as soon as every source has emitted *at least once*, and then re-emits every time any source emits again, continuously, for as long as all sources stay alive.

Q11. What does `withLatestFrom` do, and how is it different from `combineLatest`?

A: `withLatestFrom(other$)` only emits when the *source* Observable emits; at that moment it samples whatever `other$`'s latest value currently is and combines them. `other$` emitting on its own does *not* trigger a new emission — it's purely a passive "context" reader. `combineLatest`, by contrast, re-emits whenever *any* of the combined streams emits. This makes `withLatestFrom` the right choice when one stream is the "trigger" (e.g., a button click) and the other is just state to read at that instant (e.g., current form value), rather than something that should itself cause re-emission.

Q12. Why can subscribing to an Observable in a component and forgetting to unsubscribe cause a memory leak, and what are the standard ways Angular developers avoid it?

A: The source Observable (if long-lived — a `Subject` in a shared singleton service, an `interval`, `router.events`) keeps a reference to the subscriber/callback registered at `subscribe()` time. That callback typically closes over `this` (the component instance), so as long as the subscription is active, the component instance cannot be garbage collected even after Angular has destroyed and removed it from the DOM — a leak that also risks the callback executing logic against a torn-down component. Standard mitigations: the `async` pipe (Angular manages subscribe/unsubscribe automatically tied to the view), `takeUntil(this.destroy$)` with `destroy$.next()` in `ngonDestroy`, or the newer `takeUntilDestroyed()` helper (Angular 16+) which hooks into `DestroyRef` automatically.

Q13. What makes an Observable "hot" versus "cold," and give one built-in example of each.

A: Cold Observables create their producer *inside* the subscription function, so every subscriber gets an independent execution from scratch — `HttpClient.get()` is cold (two subscribes = two HTTP requests). Hot Observables share a single producer that exists independently of subscribers, so late subscribers only see emissions from the point they joined onward — `fromEvent(document, 'click')` is hot (the click events happen regardless of whether anyone is listening, and listeners only get clicks that occur after they subscribed).

Q14. How would you make a cold, expensive Observable (like an HTTP call feeding multiple components) hot and shared, and what's the difference between `share()` and `shareReplay()`?

A: Pipe it through a multicast operator. `share()` (with default config) subscribes to the source when the first subscriber arrives and unsubscribes when the last one leaves, but late subscribers get *no history* — they only see future emissions. `shareReplay(bufferSize)` additionally buffers the last `bufferSize` emissions and replays them immediately to any new/late subscriber, which is what you want for caching a completed HTTP response so multiple components requesting the same data trigger only one network call and each still gets the value even if they subscribe after it resolved.

Interviewer intent: Probing for practical caching knowledge — a candidate who only says "shareReplay caches things" without mentioning the referenced-counting/unsubscribe behavior of `share()` (and the common `shareReplay` gotcha of never unsubscribing from the source if `refCount` isn't configured) hasn't actually debugged this in production.

Q15. Why is nested `subscribe()` considered an anti-pattern, and how do you refactor it?

A: Nesting subscriptions defeats operator composition — you lose a single, unified error channel (an error in the inner subscribe won't be caught by an outer `catchError`), lose a single point of cancellation (unsubscribing the outer subscription does not automatically unsubscribe the inner one you manually created inside the callback), and it's harder to test and reason about. The fix is to flatten with a mapping operator (`switchMap` / `mergeMap` / `concatMap` depending on the desired concurrency semantics) that returns the inner Observable, letting RxJS manage the inner subscription's lifecycle as part of the single outer subscription.

Q16. What is a marble diagram, and why are they used to reason about RxJS operators?

A: A marble diagram is a visual/textual notation representing a stream over a timeline: a horizontal line represents time, `-` represents a "tick" with no emission, letters/numbers represent emitted values, `|` represents completion, and `#` represents an error. Multiple lines stacked (source, then the operator's output) let you reason precisely about *when* values are emitted relative to each other — critical for understanding operators like `debounceTime` or `switchMap` where timing, not just values, determines behavior. RxJS's own documentation and testing utilities (`TestScheduler`, marble testing via `cold()` / `hot()` helpers) use this exact notation, e.g.:

```
source:  --a--b-----c--|
debounce(20):
result:  -----b-----c|
```

Q17. How would you test a stream that uses `debounceTime` without actually waiting in real time in a unit test?

A: Use RxJS's `TestScheduler` with marble syntax, which runs "virtual time" — `debounceTime(300)` doesn't actually wait 300 real milliseconds; the scheduler advances a virtual clock synchronously so the test executes instantly while still exercising the exact timing logic. You write the input as a marble string (e.g., `'-a-b-----c|'`), the expected output as another marble string, and call

```
testScheduler.expectObservable(source$).toBe(expectedMarbles) inside testScheduler.run().
```

Q18. If you `combineLatest` two BehaviorSubjects and one of them is updated twice synchronously in the same tick, how many times does the combined stream emit?

A: Twice — `combineLatest` (like nearly all RxJS operators, absent explicit buffering) re-emits synchronously for each individual upstream emission; there's no automatic batching/coalescing of synchronous updates within the same "tick" the way, say, Angular change detection batches DOM updates. If that's undesirable, you'd need to explicitly debounce (e.g., `debounceTime(0)` to collapse to a microtask-scheduled single emission) or restructure the state updates to happen as one combined emission from the source.

7. Quick Revision Cheat Sheet

Mapping/flattening operator decision table

Need	Operator
Cancel stale requests, only latest matters (search, route param fetch)	<code>switchMap</code>

Need	Operator
Run all requests concurrently, order doesn't matter	<code>mergeMap</code>
Preserve order, run one at a time (dependent sequential calls)	<code>concatMap</code>
Ignore new triggers while one is in flight (prevent duplicate submit)	<code>exhaustMap</code>

Operator quick reference

Category	Operator	One-line purpose
Transform	<code>map</code>	1:1 value transform
Transform	<code>filter</code>	drop non-matching values
Transform	<code>tap</code>	side effect, pass value through unchanged
Rate-limit	<code>debounceTime(ms)</code>	emit after silence of <code>ms</code>
Rate-limit	<code>throttleTime(ms)</code>	emit first, then silence for <code>ms</code>
Rate-limit	<code>distinctUntilChanged()</code>	skip consecutive duplicates
Lifecycle	<code>take(n)</code> / <code>first()</code>	complete after <code>n</code> / first match
Lifecycle	<code>takeUntil(notifier\$)</code>	complete when notifier emits — put last in pipe
Error	<code>catchError(fn)</code>	replace errored stream with fallback
Error	<code>retry(n)</code>	resubscribe to source on error, <code>n</code> times
Combine	<code>combineLatest([...])</code>	re-emit combined values whenever any source emits (after all emitted once)
Combine	<code>forkJoin({...})</code>	emit once, after all sources complete, like <code>Promise.all</code>
Combine	<code>withLatestFrom(other\$)</code>	trigger from source, sample latest from <code>other\$</code>

Hot vs Cold

	Cold	Hot
Producer created	Per subscription	Once, independent of subscribers
Example	<code>http.get()</code> , <code>of()</code> , <code>new observable()</code>	<code>fromEvent()</code> , <code>Subject</code> , <code>WebSocket</code>
Late subscriber sees	Full sequence, from the start	Only future emissions (unless replayed)
Make cold → hot	<code>share()</code> / <code>shareReplay(n)</code>	—

Memory-management checklist

- Prefer the `async` pipe wherever possible — automatic subscribe/unsubscribe tied to the view.
- For manual subscriptions in components: `takeUntil(this.destroy$)` (call `destroy$.next(); destroy$.complete()` in `ngOnDestroy`) or `takeUntilDestroyed()` (Angular 16+).
- Never nest `subscribe()` calls — flatten with the appropriate mapping operator instead.
- Scope `catchError` close to the risky sub-stream so one failure doesn't permanently kill a shared/long-lived Observable.
- Cache/share expensive cold Observables consumed by multiple subscribers with `shareReplay(1)`.

Created By - Durgesh Singh